

Kernel and Scheduling

Zephyr RTOS Intermediate Course





Recap of Basic course



- What is Zephyr/Preparing dev environment
- West, west topologies, Hello World
- App Configuration: Kconfig
- App Configuration: Devicetree
- Custom Boards
- Driver Development
- Shell subsystem
- Unit Testing



About the Zephyr Intermediate Course



- Kernel & Scheduling
- Synchronization & Shared-State Safety
- Deferred Execution & Workqueues
- Communication & Event-Driven Architecture
- Memory, Resource Constraints & Reliability
- Tracing, Debugging & Runtime Analysis
- Ztest
- Security



Today's Agenda

- Execution model & thread states
- Priority model & scheduler mechanics
- Priority inversion & latency
- Design rules + live demos





Execution Model



Execution context in Zephyr



Threads	Interrupts (ISRs)
Managed by scheduler	Triggered by hardware
Have priority, state, stack	Bypass scheduler, run immediately
Can sleep, block, yield	Cannot block or wait
Preemptible by higher priority	Can preempt any thread
Cooperate with kernel scheduling	Execute with minimal latency



Execution Is Not Sequential



- What code looks like:

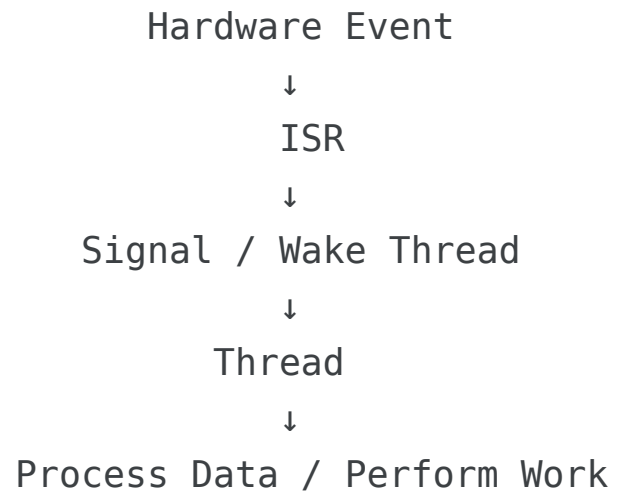
```
do_step1();  
do_step2();
```

- What may actually happen:

```
do_step1();  
  
<interrupt occurs>  
<ISR wakes higher-priority thread>  
<scheduler switches context>  
  
do_step2();
```



Design Pattern



- **ISRs notify**
- **Threads process**



Thread Model



Creating Threads

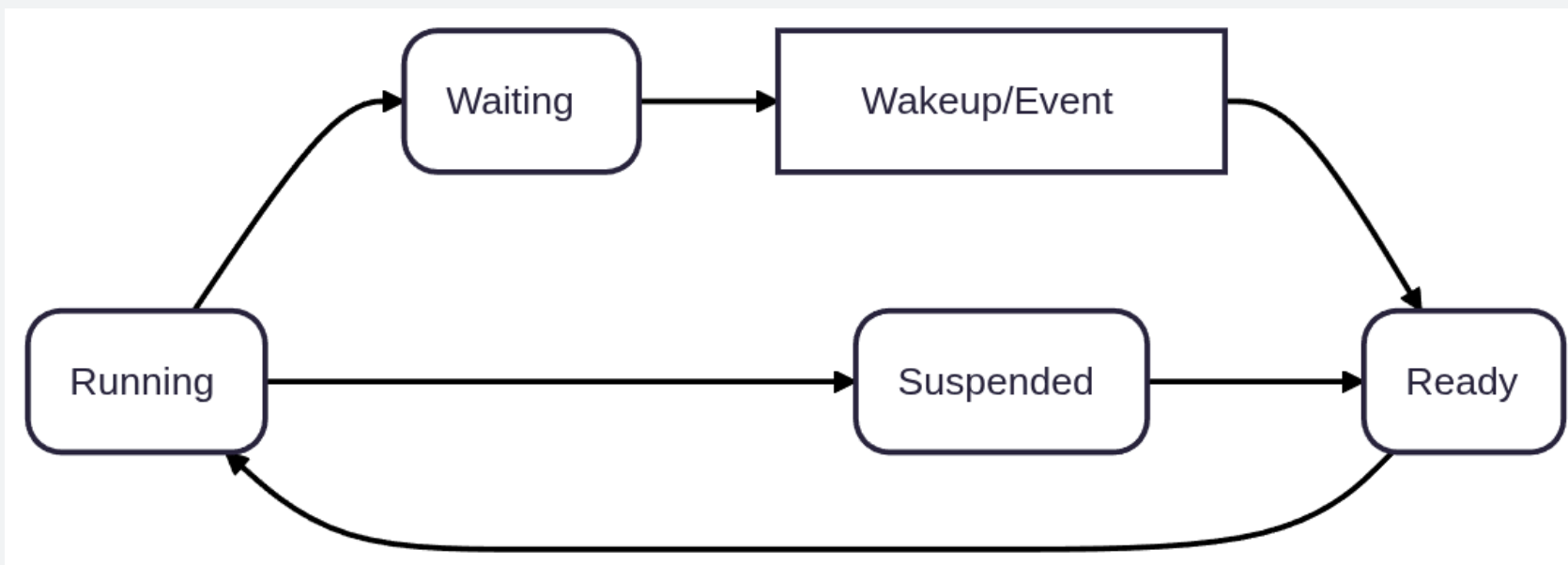


```
K_THREAD_DEFINE (name, stack_size, entry, p1, p2, p3, prio, options, delay);
```

- `name`: Name of the thread
- `stack_size`: Stack size in bytes
- `entry`: Thread entry function
- `p1, p2, p3`: Entry point parameters
- `prio`: Thread priority
- `options`: Thread options
- `delay`: Scheduling delay (in milliseconds), zero for no delay



Thread Lifecycle



- **Ready:** eligible to run
- **Running:** currently executing
- **Waiting:** paused until an event
- **Suspended:** explicitly paused



Yielding and Waiting



```
k_yield();           // give CPU to another Ready thread  
k_sleep(K_MSEC(100)); // put thread to Waiting state
```

Key difference:

- `k_yield()` - Thread stays **Ready** but lets another thread run
- `k_sleep()` - Thread enters **Waiting** state until timeout expires



Demo 1: Thread Interleaving



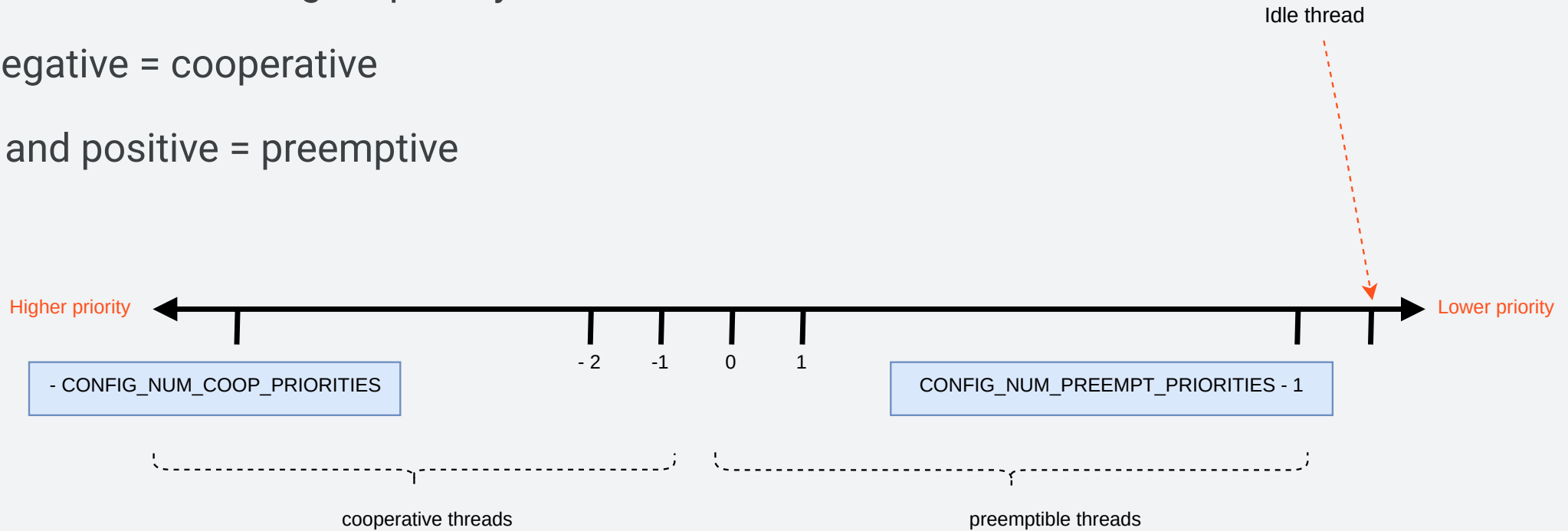
Priority Model



Priority System



- Lower number = higher priority
- Negative = cooperative
- 0 and positive = preemptive





Preemptive vs Cooperative Threads



Cooperative	Preemptive
Keeps running until it waits or yields	Can be suspended when a higher/equal-priority thread becomes ready
Another thread cannot preempt it	Scheduler may switch immediately
Good for short, controlled sections	Good for normal application work

- **ISRs** can interrupt both types



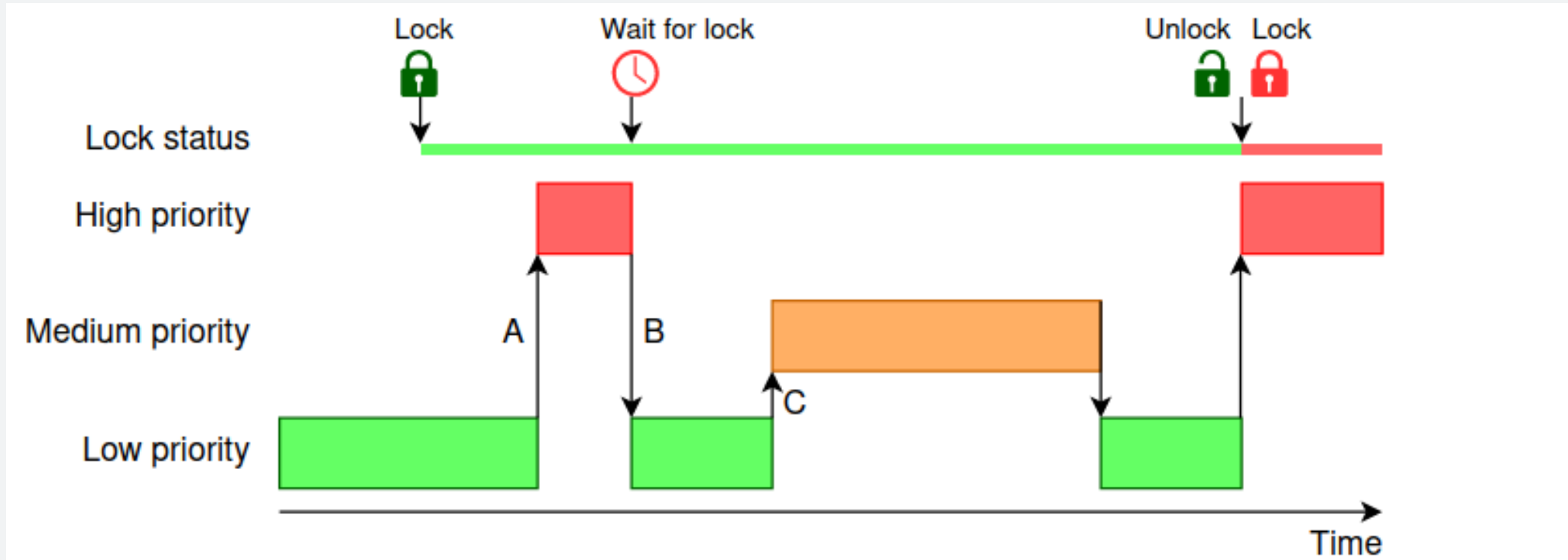
Demo 2: Scheduling Competition



Timing Effects of Priority



Priority Inversion





Latency and Responsiveness



Event

↓

ISR entry → interrupt latency

↓

ISR exit

↓

Thread becomes READY

↓

Thread executes → scheduling latency

↓

Application response → end-to-end latency

- **Latency** is how priority inversion and scheduling decisions become visible in real time



Practical Takeaway



- Keep critical sections short
- Avoid long-running work while holding shared resources
- Prefer designs that reduce contention
- Use blocking instead of busy loops
- Remember: a thread that never WAITs can hurt system responsiveness



Home task - Scheduler Behavior Explorer



- Goal: Build working intuition for Ready vs Waiting and `k_yield` vs `k_sleep`
- Task: Create three preemptive threads at different priorities:
 - `t_low_fn` (priority 7): logs "T_LOW running", sleeps 300ms
 - `t_med_fn` (priority 5): logs "T_MED running", sleeps 200ms
 - `t_high_fn` (priority 3): logs "T_HIGH running", sleeps 100ms
- Observe: Which thread runs most often? Does priority affect interleaving?
- Add a fourth cooperative thread at priority -1 that does 5 iterations of busy work, then yields
- Push tag: `l1-task1`

Thank You





Resources

1. [Zephyr Threads](#)
2. [Zephyr Scheduling](#)

